

Linux Firewall, Network Security and Log Management

Description

1. Learn deeply about different types of logs in Linux systems, real-time monitoring of logs and journals.
Note: Logs are available under `/var/log/`.
2. Install and configure commonly used firewalls such as CSF and APF. Try to manage different services via the same.
3. Learn management of incoming/outgoing services to an AWS instance with security groups.
4. Learn external server attacks and prevention parameters, e.g. phishing, brute force.

Linux Logs and Real-Time Monitoring

1.1 Introduction

Linux systems maintain logs that record important information about system activity, services, authentication, errors, and security events.

Most traditional Linux log files are stored under:

`/var/log/`

These logs help administrators troubleshoot problems, monitor services, investigate security events, and understand what is happening on a server.

1.2 Important Types of Linux Logs

System Logs

System logs contain information about general system activity, services, kernel messages, and system events.

Common locations include:

`/var/log/messages`

`/var/log/syslog`

The exact files depend on the Linux distribution and logging configuration.

Authentication Logs

Authentication logs record login attempts, SSH activity, sudo usage, and other authentication-related events.

On RHEL-based systems, authentication information can commonly be found in:

`/var/log/secure`

These logs are useful for identifying failed login attempts and suspicious authentication activity.

Boot Logs

Boot-related information helps identify problems that occur while the operating system starts.

Examples include:

`/var/log/boot.log`

and systemd journal information.

Application Logs

Applications and services can maintain their own log files.

For example:

`/var/log/httpd/`

contains Apache HTTP Server logs.

Common Apache logs include:

```
access_log  
error_log
```

The access log records requests received by the web server, while the error log records problems encountered by Apache.

Firewall Logs

Firewall systems can generate logs showing allowed, blocked, or suspicious network traffic.

For example, CSF/LFD can maintain:

```
/var/log/lfd.log
```

These logs can be used to investigate firewall and security events.

1.3 Viewing Logs

A log file can be viewed using:

```
cat /var/log/messages
```

For large files, less is more convenient:

```
less /var/log/messages
```

Specific entries can be searched using:

```
grep "error" /var/log/messages
```

1.4 Real-Time Log Monitoring

The tail command can be used to monitor a log file as new entries are added.

```
sudo tail -f /var/log/messages
```

The -f option continuously follows the file and displays new log entries in real time.

For Apache:

```
sudo tail -f /var/log/httpd/access_log
```

This can be useful while accessing the website because new HTTP requests appear in the log.

1.5 Journal Monitoring

Modern RHEL systems use **systemd-journald** to collect system and service logs.

The journal can be viewed with:

```
sudo journalctl
```

To monitor the journal in real time:

```
sudo journalctl -f
```

To view logs for a particular service:

```
sudo journalctl -u httpd
```

To follow a service's logs continuously:

```
sudo journalctl -u httpd -f
```

Result

Linux logs provide information about system activity, authentication, applications, services, and security events. Real-time monitoring using `tail -f` and `journalctl -f` allows administrators to observe events as they happen.

Task 2 – Installation and Configuration of CSF and APF

2.1 CSF – ConfigServer Security & Firewall

CSF is a Linux firewall management and security tool used to control network traffic.

Important CSF Features

- Incoming traffic control
- Outgoing traffic control
- Port management
- IP allow/deny
- IP whitelisting
- IP blacklisting
- Temporary IP blocking
- Connection protection
- Brute-force protection through LFD
- Firewall logging

2.2 CSF Configuration

The main configuration file is:

```
/etc/csf/csf.conf
```

Other important files include:

```
/etc/csf/csf.allow  
/etc/csf/csf.deny  
/etc/csf/csf.ignore  
/etc/csf/csf.pignore
```

The main TCP and UDP settings include:

```
TCP_IN  
TCP_OUT  
UDP_IN  
UDP_OUT
```

2.3 CSF Installation

CSF was installed on the RHEL 10.2 AWS EC2 instance.

The required Perl packages were installed:

```
sudo dnf install -y perl perl-core
```

CSF was then successfully installed.

The RHEL `firewalld` service was disabled so that CSF could manage the firewall rules without another firewall manager competing with it.

2.4 CSF Testing

The CSF testing mode was disabled before activating the firewall:

```
TESTING = "0"
```

The current SSH IP was added to the CSF allow list to prevent the SSH connection from being blocked.

An IP blocking test was performed:

```
sudo csf -d 1.1.1.1 "CSF test block"
```

The test entry was then removed:

```
sudo csf -dr 1.1.1.1
```

This demonstrated CSF's ability to deny and remove IP addresses.

2.5 Managing Apache with CSF

Apache was enabled:

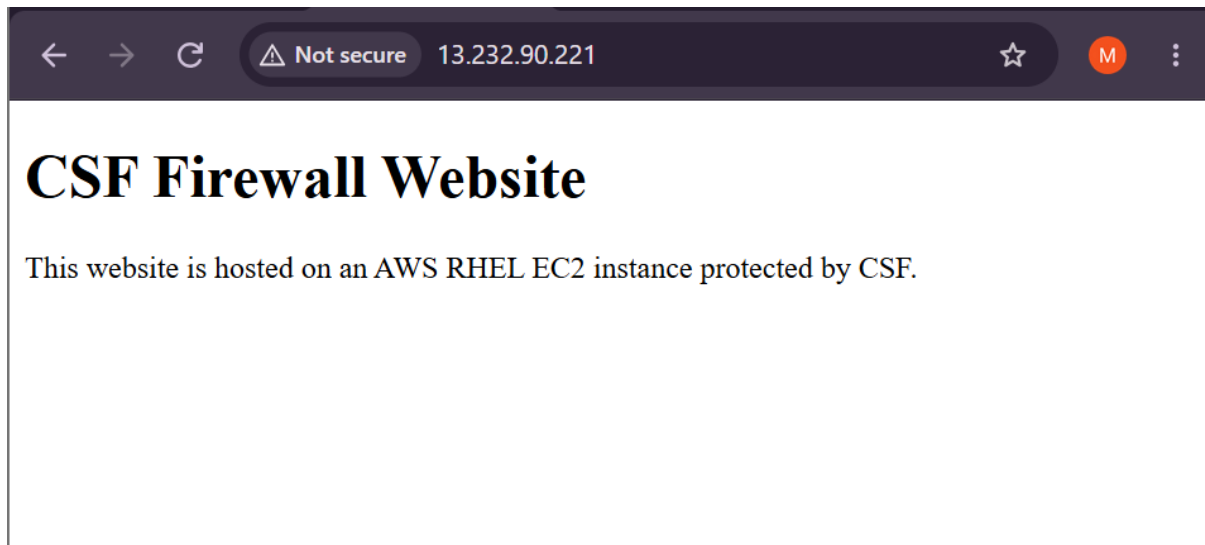
```
sudo systemctl enable --now httpd
```

A test webpage was created under:

```
/var/www/html/index.html
```

HTTP port 80 was allowed through CSF.

The webpage was successfully accessed using the EC2 instance's public IP address.



Practical Authentication Log Test – Failed User Login

A test was performed to verify that failed login attempts are recorded in the Linux authentication logs.

A fake/non-existing user “rahul” was used to attempt an SSH login to the RHEL EC2 instance. The login attempt was rejected with:

Permission denied

The authentication log was then checked:

```
sudo cat /var/log/secure
```

The failed login attempt was visible in `/var/log/secure`, confirming that Linux records authentication-related events.

This demonstrated the practical relationship between:

```
SSH Login Attempt
    ↓
Authentication Failure
    ↓
Permission denied
    ↓
/var/log/secure
    ↓
Security investigation
```

Screenshot Output

```
[ec2-user@ip-172-31-6-111 ~]$ sudo tail -20 /var/log/secure
Sep 23 06:55:28 ip-172-31-6-111 sudo[3183]: ec2-user : TTY=pts/0 ; PWD=/home/ec2-user ; USER=root ; COMMAND=/bin/systemctl enable --now httpd
Sep 23 06:55:28 ip-172-31-6-111 sudo[3183]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 06:55:29 ip-172-31-6-111 sudo[3183]: pam_unix(sudo:session): session closed for user root
Sep 23 06:55:37 ip-172-31-6-111 sudo[3422]: ec2-user : TTY=pts/0 ; PWD=/home/ec2-user ; USER=root ; COMMAND=/bin/systemctl status httpd --no-pager
Sep 23 06:55:37 ip-172-31-6-111 sudo[3422]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 06:55:37 ip-172-31-6-111 sudo[3422]: pam_unix(sudo:session): session closed for user root
Sep 23 06:58:42 ip-172-31-6-111 sudo[3462]: ec2-user : TTY=pts/0 ; PWD=/home/ec2-user ; USER=root ; COMMAND=/bin/vi /var/www/html/index.html
Sep 23 06:58:42 ip-172-31-6-111 sudo[3462]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 06:58:49 ip-172-31-6-111 sudo[3462]: pam_unix(sudo:session): session closed for user root
Sep 23 06:59:25 ip-172-31-6-111 sudo[3468]: ec2-user : TTY=pts/0 ; PWD=/home/ec2-user ; USER=root ; COMMAND=/bin/cat /var/www/html/index.html
Sep 23 06:59:25 ip-172-31-6-111 sudo[3468]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 06:59:25 ip-172-31-6-111 sudo[3468]: pam_unix(sudo:session): session closed for user root
Sep 23 06:59:47 ip-172-31-6-111 sudo[3476]: ec2-user : TTY=pts/0 ; PWD=/home/ec2-user ; USER=root ; COMMAND=/sbin/csf -p 80
Sep 23 06:59:47 ip-172-31-6-111 sudo[3476]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 06:59:47 ip-172-31-6-111 sudo[3476]: pam_unix(sudo:session): session closed for user root
Sep 23 07:04:24 ip-172-31-6-111 sshd-session[3608]: Invalid user rahul from 49.47.198.161 port 50579
Sep 23 07:04:24 ip-172-31-6-111 sshd-session[3608]: Connection closed by invalid user rahul 49.47.198.161 port 50579 [preauth]
Sep 23 07:07:12 ip-172-31-6-111 sudo[3641]: ec2-user : TTY=pts/0 ; PWD=/home/ec2-user ; USER=root ; COMMAND=/bin/tail -20 /var/log/1fd.log
Sep 23 07:07:12 ip-172-31-6-111 sudo[3641]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 07:07:12 ip-172-31-6-111 sudo[3641]: pam_unix(sudo:session): session closed for user root
[ec2-user@ip-172-31-6-111 ~]$
```

Result

The test successfully demonstrated that a failed login attempt can be detected and investigated using the Linux authentication log. This is useful for identifying repeated unauthorized login attempts and potential brute-force activity.

2.6 APF – Advanced Policy Firewall

APF is a Linux firewall management tool used to control incoming and outgoing network traffic according to configured policies.

Important APF Features

- Inbound traffic control
- Outbound traffic control
- TCP/UDP port management
- IP filtering
- Port-based filtering
- Network interface configuration
- Custom firewall policies
- Logging
- Service protection

2.7 APF Installation

APF version 2.0.1 was installed on the RHEL EC2 instance.

The main configuration file is:


```
/etc/apf/conf.apf
```

The APF executable is:

```
/usr/local/sbin/apf
```

The EC2 network interface was identified as:

```
ens5
```

Therefore, APF was configured with:

```
IFACE_UNTRUSTED="ens5"
```

2.8 APF Port Configuration

SSH and HTTP were allowed:

```
IG_TCP_CPORTS="22,80"
```

This allowed:

- Port **22** – SSH
- Port **80** – HTTP

APF was started with:

```
sudo /usr/local/sbin/apf -s
```

APF successfully initialized and reported:

```
opening inbound tcp port 22
opening inbound tcp port 80
default (egress) output accept
default (ingress) input drop
firewall initialized
```

2.9 Managing Apache with APF

The Apache webpage was changed to identify APF:

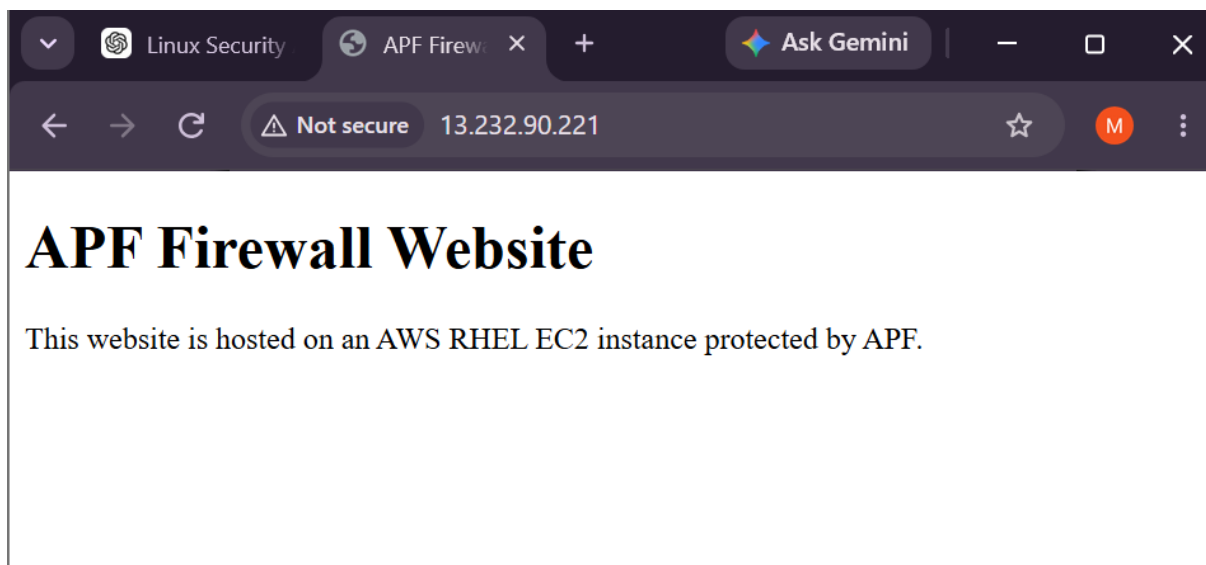
<h1>APF Firewall Website</h1>

<p>This website is hosted on an AWS RHEL EC2 instance protected by APF.</p>

The webpage was successfully accessed through the EC2 public IP address while APF was active.

Result

The practical test demonstrated that APF could protect the EC2 instance while allowing the required HTTP service through port 80.



Practical Authentication Log Test – Failed User Login

A test was performed to verify that failed login attempts are recorded in the Linux authentication logs.

A fake/non-existing user “raj” was used to attempt an SSH login to the RHEL EC2 instance. The login attempt was rejected with:

Permission denied

The authentication log was then checked:

```
sudo cat /var/log/secure
```

The failed login attempt was visible in `/var/log/secure`, confirming that Linux records authentication-related events.

This demonstrated the practical relationship between:

SSH Login Attempt
↓
Authentication Failure
↓
Permission denied
↓
/var/log/secure
↓
Security investigation

Screenshot Output

```
[ec2-user@ip-172-31-6-111 apf-2.0.1]$ sudo tail -10 /var/log/secure
Sep 23 07:22:28 ip-172-31-6-111 sshd-session[6123]: Invalid user rahul from 49.47.198.161 port 60764
Sep 23 07:22:28 ip-172-31-6-111 sshd-session[6123]: Connection closed by invalid user rahul 49.47.198.161 port 60764 [preauth]
Sep 23 07:23:45 ip-172-31-6-111 sudo[6126]: ec2-user : TTY=pts/0 ; PWD=/tmp/apf-2.0.1 ; USER=root ; COMMAND=/bin/tail -20 /var/log/secure
Sep 23 07:23:45 ip-172-31-6-111 sudo[6126]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 07:23:45 ip-172-31-6-111 sudo[6126]: pam_unix(sudo:session): session closed for user root
Sep 23 07:24:12 ip-172-31-6-111 sudo[6130]: ec2-user : TTY=pts/0 ; PWD=/tmp/apf-2.0.1 ; USER=root ; COMMAND=/bin/tail -10 /var/log/secure
Sep 23 07:24:12 ip-172-31-6-111 sudo[6130]: pam_unix(sudo:session): session opened for user root(uid=0) by ec2-user(uid=1000)
Sep 23 07:24:12 ip-172-31-6-111 sudo[6130]: pam_unix(sudo:session): session closed for user root
Sep 23 07:28:05 ip-172-31-6-111 sshd-session[6112]: Invalid user raj from 49.47.198.161 port 51500
Sep 23 07:28:05 ip-172-31-6-111 sshd-session[6112]: Connection closed by invalid user raj 49.47.198.161 port 51500 [preauth]
[ec2-user@ip-172-31-6-111 apf-2.0.1]$
```

Result

The test successfully demonstrated that a failed login attempt can be detected and investigated using the Linux authentication log. This is useful for identifying repeated unauthorized login attempts and potential brute-force activity.

Task 3 – Managing AWS Instance Services Using Security Groups

3.1 Introduction

An AWS **Security Group** acts as a virtual firewall for an EC2 instance.

It controls:

- Incoming traffic through **inbound rules**
- Outgoing traffic through **outbound rules**

Security Groups operate at the AWS network level, before traffic reaches the operating system's firewall.

3.2 Inbound Rules

Inbound rules determine which traffic can reach the EC2 instance.

For a web server, common rules are:

Type	Port	Purpose
SSH	22	Remote administration
HTTP	80	Web traffic
HTTPS	443	Secure web traffic

SSH access should normally be restricted to the required source IP rather than allowing unnecessary sources.

3.3 Outbound Rules

Outbound rules control traffic leaving the EC2 instance.

Examples include:

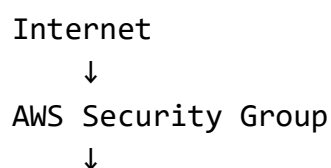
- HTTP – 80
- HTTPS – 443
- DNS – 53
- Other required services

These rules determine whether the instance can communicate with external services.

3.4 Security Group and Linux Firewall

AWS Security Groups and Linux firewalls such as CSF/APF operate at different layers.

The traffic flow can be understood as:



EC2 Instance
↓
Linux Firewall
↓
Application / Service

For example, HTTP traffic requires:

AWS Security Group → TCP 80 allowed
↓
Linux Firewall → TCP 80 allowed
↓
Apache → Port 80

If either the Security Group or Linux firewall blocks the traffic, the webpage will not be accessible.

3.5 Service Management

Different services can be controlled by allowing or restricting their corresponding ports.

Examples:

SSH	→ 22
HTTP	→ 80
HTTPS	→ 443
DNS	→ 53
FTP	→ 21

Only the ports required by the application should be exposed.

Task 4 – External Server Attacks and Prevention

4.1 Introduction

Internet-facing servers can be exposed to different types of attacks. Understanding common attacks helps administrators configure appropriate security controls.

4.2 Phishing

Phishing is a social-engineering attack in which an attacker attempts to trick users into providing sensitive information such as:

- Usernames
- Passwords
- Banking information
- Authentication codes
- Other confidential information

Phishing commonly uses fake emails, websites, or messages that appear to come from a trusted source.

Prevention

- Do not open suspicious links or attachments.
- Verify the sender.
- Use multi-factor authentication.
- Use secure passwords.
- Educate users about phishing.
- Use email and web security controls.

4.3 Brute-Force Attack

A brute-force attack attempts to gain access by repeatedly trying different passwords or credentials.

SSH servers are common targets for automated login attempts.

Prevention

Important measures include:

- Use strong passwords.
- Use SSH keys instead of password authentication where appropriate.
- Restrict SSH access through AWS Security Groups.
- Restrict SSH access through Linux firewalls.

- Monitor authentication logs.
- Use tools such as CSF/LFD for login-failure monitoring.
- Limit repeated authentication attempts.
- Disable unnecessary services.

4.4 Port Scanning

Attackers may scan a server to discover open ports and identify available services.

For example, an attacker may attempt to determine whether ports such as:

22
80
443
3306

are accessible.

Prevention

- Close unnecessary ports.
- Allow only required services.
- Configure AWS Security Groups carefully.
- Configure the Linux firewall.
- Monitor firewall logs.
- Keep services updated.

4.5 Denial-of-Service Attacks

A denial-of-service attack attempts to make a service unavailable by sending excessive traffic or requests.

Prevention

Depending on the attack, protection can include:

- Firewall rules
- Rate limiting
- Connection limits

- AWS security controls
- Monitoring
- Load balancing
- DDoS protection services

4.6 Service Vulnerabilities

An attacker may exploit vulnerabilities in an outdated service or application.

Examples include vulnerable:

- Web servers
- Databases
- Operating systems
- Web applications
- Network services

Prevention

- Keep the operating system updated.
- Update installed applications.
- Remove unnecessary services.
- Restrict network access.
- Monitor logs.
- Use security tools and firewalls.

4.7 Security Monitoring

Security logs should be monitored regularly for suspicious activity.

Important sources include:

```
/var/log/secure  
/var/log/messages  
/var/log/httpd/
```

For systemd services:

```
sudo journalctl
```


For real-time monitoring:

```
sudo journalctl -f
```

Firewall tools such as CSF can also provide security-related logging.

Conclusion

In this assignment, the concepts of **Linux log management, firewall configuration, AWS Security Groups, and server attack prevention** were studied.

Linux logs under `/var/log/` and the systemd journal provide information about system, authentication, application, and security activity. Real-time monitoring can be performed using tools such as `tail` and `journalctl`.

CSF and APF were practically installed and configured on an AWS RHEL EC2 instance.

Apache HTTP was used as a service to demonstrate firewall-based service management. SSH and HTTP ports were configured to remain accessible while the firewall was active.